Reviewer Pack — Kronecker Coefficient Exponential Gap in Symmetric Powers of…

Entry 02a5c9227ac1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 15:27:59 UTC

Kronecker Coefficient Exponential Gap in Symmetric Powers of Permanent vs Determinant

Entry ID: `02a5c9227ac1` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 15:27:59 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Group **Field B** (complexity object): Boolean Circuit Size

Statement:

> For any CNF formula Φ with n variables, the Kronecker coefficient of the tensor product of the symmetric power S^k(ρ) and S^m(ρ) (where ρ is the standard representation of S_n) is exponentially larger for the permanent than for the determinant when k = n^{0.6} and m = n^{0.4}. This exponential gap implies that the circuit size of Φ is at least 2^{Ω(n)}.

Rationale (proposer's reasoning):

> Kronecker coefficients capture the multiplicity of irreducible representations in tensor products, which could reflect the structural complexity of the CNF. The exponential gap in these coefficients might indicate that the CNF's structure requires exponentially larger circuits, aligning with the known hardness of the permanent.

Taxonomy category: `ACC_LB_via_WILLIAMS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9226131b70405590`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM_1	LLM_2
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Parameters for the trial
    n = 10  # Number of variables in the CNF formula
    k = int(n ** 0.6)
    m = int(n ** 0.4)

    # Generate a random CNF formula with n variables and some clauses
    num_clauses = 2 * n
    cnf = []
    for _ in range(num_clauses):
        clause = [random.randint(1, n), -random.randint(1, n)]
        cnf.append(clause)

    # Function to compute the Kronecker coefficient (simplified version)
    def kronecker_coefficient(k, m):
        if k == 0 or m == 0:
            return 1
        return math.comb(k + m - 1, k) / math.comb(k + m, k)

    # Compute Kronecker coefficients for permanent and determinant cases
    perm_coeff = kronecker_coefficient(k, m)
    det_coeff = kronecker_coefficient(k, m)

    # Check if the exponential gap holds
    if perm_coeff > 2 * det_coeff:
        conjecture_holds = True
        counterexample = ""
    else:
        conjecture_holds = False
        counterexample = "Exponential gap not observed"

    return {
        "metric_name": "Kronecker Coefficient Exponential Gap",
```

```
        "metric_value": perm_coeff / det_coeff,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30))  # Default to f

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results]
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(metric_values)/len(metric_values):.2f} std=0 support_f
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Exponential gap not observed\" first_failing_se
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ture_holds': False, 'counterexample': 'Exponential gap not observed'}
TRIAL: {'metric_name': 'Kronecker Coefficient Exponential Gap', 'metric_value': 1.0, 'instances_teste
TRIAL: {'metric_name': 'Kronecker Coefficient Exponential Gap', 'metric_value': 1.0, 'instances_teste
TRIAL: {'metric_name': 'Kronecker Coefficient Exponential Gap', 'metric_value': 1.0, 'instances_teste
TRIAL: {'metric_name': 'Kronecker Coefficient Exponential Gap', 'metric_value': 1.0, 'instances_teste
TRIAL: {'metric_name': 'Kronecker Coefficient Exponential Gap', 'metric_value': 1.0, 'instances_teste
TRIAL: {'metric_name': 'Kronecker Coefficient Exponential Gap', 'metric_value': 1.0, 'instances_teste
TRIAL: {'metric_name': 'Kronecker Coefficient Exponential Gap', 'metric_value': 1.0, 'instances_teste
TRIAL: {'metric_name': 'Kronecker Coefficient Exponential Gap', 'metric_value': 1.0, 'instances_teste

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ec13860c.py", line 75, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ec13860c.py", line 75, in <genexpr>
    first_failing_seed = next(r["see
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed before producing reliable results, and the single trial's counterexample may not be statistically significant. | next: Run the test with multiple instances and ensure proper seed handling to validate the conjecture

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	99681
2	propose	ollama_remote	qwen3:8b	0	0	112071
3	propose	ollama_remote	qwen3:8b	0	0	111923
4	preregistration	ollama_remote	qwen3:8b	0	0	24302
5	novelty	ollama_remote	qwen3:8b	0	0	20670
6	novelty	ollama_remote	qwen3:8b	0	0	16536
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16572
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7091
9	judge	ollama_remote	qwen3:8b	0	0	17082

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 425927 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/02a5c9227ac1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/02a5c9227ac1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/02a5c9227ac1.tar.gz` (if generated)